

Maximum Sum Subarray of Size K

Two approaches, visualized execution, and the core logic to remember

Brute Force · $O(n \cdot k)$

Sliding Window · $O(n)$

Reference notes · MaxSumSubarrayOfSizeK.java

Problem Statement

Given an array `arr[]` and a number `k`, return the maximum sum of any contiguous subarray of size `k`.

`arr =`
`[1,4,2,10,23,3,1,0,20]`
`k = 4`

→

find max sum
of any size-4 window

→

Output: 39
(4+2+10+23)

Approach Comparison

Approach	Time	Space	Core Idea
Brute Force	$O(n \cdot k)$	$O(1)$	Recompute the full window sum at every starting index
Sliding Window	$O(n)$	$O(1)$	Reuse the previous window sum — subtract left, add right

Big picture: Consecutive windows of size `k` overlap in `k - 1` elements. Brute force throws that overlap away and recomputes from scratch every time. Sliding window reuses it — each step becomes an $O(1)$ update instead of an $O(k)$ resum.

1 · Brute Force

$O(n \cdot k)$ time

$O(1)$ space

For each
start index i



Sum next
 k elements



Track
running max

```
// recompute the whole window sum at every starting position
public static int bruteForce(int[] arr, int k) {
    int maxSum = 0;

    for (int i = 0; i <= arr.length - k; i++) {
        int currentSum = 0;

        for (int j = i; j < i + k; j++) {
            currentSum += arr[j];
        }

        maxSum = Math.max(maxSum, currentSum);
    }
    return maxSum;
}
```

Recall trick: "Recompute the whole window sum every time." Correct, but re-adds k elements at every starting position — wasteful, since consecutive windows overlap in $k-1$ elements.

2 · Sliding Window

O(n) time

O(1) space

Sum first
window (0..k-1)



Track max
so far



Subtract left,
add right



high==length?
stop

```
// slide, don't recompute: drop the left element, add the right element
public static int usingSlidingWindow(int[] arr, int k) {
    int resultSum = 0;
    int currentSum = 0;
    int low = 0;
    int high = k - 1;

    // first window calculation
    for (int i = 0; i < k; i++) {
        currentSum += arr[i];
    }

    while (high < arr.length) {
        resultSum = Math.max(resultSum, currentSum);
        low++;
        high++;

        if (high == arr.length) {
            break;
        }
        currentSum = currentSum - arr[low - 1] + arr[high];
    }
    return resultSum;
}
```

Trace for arr = [1,4,2,10,23,3,1,0,20], k = 4:

low	high	currentSum	resultSum after max	next currentSum
0	3	1+4+2+10=17	17	17−arr[0]+arr[4]=39
1	4	39	39	39−arr[1]+arr[5]=38
2	5	38	39	38−arr[2]+arr[6]=37
3	6	37	39	37−arr[3]+arr[7]=27
4	7	27	39	27−arr[4]+arr[8]=24
5	8	24	39	high==length → break

Recall trick: "Slide, don't recompute: drop the left element, add the right element." The `if (high == arr.length) break;` guard is the subtle but essential piece — without it, `arr[high]` would be read out of bounds right after the last valid window.

TL;DR — For Future Me

What to remember

- **Best solution:** Sliding window, O(n) — "drop the element leaving on the left, add the element entering on the right — never recompute the whole sum."
- **Brute force** is O(n·k) — correct but wasteful, since it ignores the overlap between consecutive windows.
- **Watch for:** the `high == arr.length` guard right after sliding the window — without it, the next update reads past the end of the array on the final iteration.
- **Underlying pattern:** the canonical fixed-size sliding window template — compute the first window directly, then maintain it incrementally. Reused constantly in "max/min sum of size-k subarray," "longest substring with constraints," and "average of subarrays of size k" problems.